

M0 · SECURITY REVIEW

M0 USD8.

A security review of USD8.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
M0	USD8	Guardian	November 28 – November 29, 2025	December 8, 2025

§ 00

Contents.

19 pages — overview, confidence ranking, scope, 10 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 10 findings	P. 08
§ 05	Disclaimer	P. 18
§ 06	About Guardian	P. 19

§ 01

Overview.

Guardian reviewed USD8 across a single review round. The final report includes 1 High, 1 Low, 8 Informational findings.

AUDIT FIRM	Guardian	REPOSITORY	mocaverse/USD8
CLIENT	M0	METHODOLOGY	Smart contract · Manual · Main Review
PROJECT	USD8	MAIN REVIEW COMMIT	2a0b6d37be26
PREPARED BY	Guardian		
AUDIT WINDOW	November 28 – November 29, 2025		
FINAL REPORT	December 8, 2025		

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	1	0	0	0	1
Medium	0	0	0	0	0
Low	1	0	0	0	1
Informational	8	0	4	0	4
All severities	10	0	4	0	6

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed USD8 in `mocaverse/USD8`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Contracts Do Not Compile	Typo	HIGH	RESOLVED	p. 8
L-01	setYieldRecipient Not Always Callable	DoS	LOW	RESOLVED	p. 9
I-01	Outdated Initialize NatSpec	Documentation	INFORMATIONAL	RESOLVED	p. 10
I-02	Protocol Fee Amount Rounded Down	Rounding	INFORMATIONAL	ACKNOWLEDGED	p. 11
I-03	Unnecessary _beforeClaimYield Override	Superfluous Code	INFORMATIONAL	RESOLVED	p. 12
I-04	Misleading Documentation	Documentation	INFORMATIONAL	RESOLVED	p. 13
I-05	Multiple Initialization Steps Are Necessary	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 14
I-06	claimYield Bypasses Freeze List	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 15
I-07	Multiple Initialize Functions Risk	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 16
I-08	Outdated MUSD References	Documentation	INFORMATIONAL	RESOLVED	p. 17

HIGH **H-01**

RESOLVED

Contracts Do Not Compile

CATEGORY	Typo
LOCATION	USD8.sol
REVIEW	Main Review

DESCRIPTION

In the claimYield function the hasRole check has been replaced with `sl` which does not compile.

RECOMMENDATION

Correct the typo.

RESOLUTION

Status: Resolved.

LOW **L-01**

RESOLVED

setYieldRecipient Not Always Callable

CATEGORY	DoS
LOCATION	USD8.sol
REVIEW	Main Review

— DESCRIPTION

MYieldToOne.setYieldRecipient() unconditionally calls claimYield() before updating the recipient. USD8's claimYield reverts if currentYield == 0 and also reverts when fee > 0 but protocolFeeAmount rounds to 0 (dust), and it is additionally blocked when paused. This means setYieldRecipient will revert under common conditions (no yield yet, small yield, or paused), preventing administrators from changing the yield destination.

— RECOMMENDATION

Consider refactoring this flow to early return in these cases instead of reverting to allow this configuration to occur.

— RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

RESOLVED

Outdated Initialize NatSpec

CATEGORY Documentation
LOCATION USD8.sol
REVIEW Main Review

DESCRIPTION

In the NatSpec for the initialize function there is no NatSpec for the `forcedTransferManager` parameter.

RECOMMENDATION

Update the NatSpec for the initialize function to include the `forcedTransferManager` parameter.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

ACKNOWLEDGED

Protocol Fee Amount Rounded Down

CATEGORY	Rounding
LOCATION	USD8.sol
REVIEW	Main Review

DESCRIPTION

In the 4th scenario in the `claimYield` function the `protocolFeeAmount` is computed using round down division, rounding against the protocol and in favor of the `yieldRecipient`.

This however poses almost no impact since the `claimYield` function is permissioned to be callable only by the admin or the `YIELD_RECIPIENT_MANAGER_ROLE` and normal usage will only truncate a handful of wei in favor of the yield recipient over the protocol recipient over time.

RECOMMENDATION

Be aware that the existing protocol fee logic rounds against the protocol and in favor of the yield recipient. If desired, consider using round up logic for the protocol fee amount, however keeping the code unchanged and using division by truncation, while acknowledging this behavior, is more straightforward.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

RESOLVED

Unnecessary `_beforeClaimYield` Override

CATEGORY	Superfluous Code
LOCATION	USD8.sol
REVIEW	Main Review

DESCRIPTION

In the USD8 contract the `_beforeClaimYield` function is unnecessarily overridden to add the `onlyRole(YIELD_RECIPIENT_MANAGER_ROLE)` modifier, however the entire `claimYield` function has been overridden in the USD8 contract, which entirely bypasses this `_beforeClaimYield` hook.

RECOMMENDATION

Either invoke the `_beforeClaimYield` hook instead of the authorization that is performed at the beginning of the `claimYield` function, or remove the `_beforeClaimYield` override entirely.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-04

RESOLVED

Misleading Documentation

CATEGORY	Documentation
LOCATION	USD8.sol
REVIEW	Main Review

DESCRIPTION

In the NatSpec for the `_beforeClaimYield` function it is mentioned that `Addresses with the YIELD_RECIPIENT_MANAGER_ROLE` are still able to claim yield when the contract is paused.

However the entire `claimYield` function has been overridden and uses a `whenNotPaused` modifier, thus contradicting this comment.

RECOMMENDATION

Clarify what the desired behavior is and if it is desired that certain roles can claim yield even when the system is paused consider removing the `whenNotPaused` modifier.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-05

ACKNOWLEDGED

Multiple Initialization Steps Are Necessary

CATEGORY	Warning
LOCATION	USD8.sol
REVIEW	Main Review

DESCRIPTION

Typically the `initialize` function handles the entire initialization of a contract that is necessary. However the initialize function for the USD8 contract does not instantiate an initial protocol treasury address and therefore a treasury must always be set with the `setProtocolTreasuryAddress` function before yield can be claimed.

RECOMMENDATION

Confirm if this secondary initialization step is desired, if not consider including an initial treasury address in the initialize function.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-06**

ACKNOWLEDGED

claimYield Bypasses Freeze List

CATEGORY	Warning
LOCATION	USD8.sol
REVIEW	Main Review

— DESCRIPTION

claimYield mints tokens to the yieldRecipient and protocolTreasuryAddress without checking if these accounts are frozen.

This bypasses the freeze policy applied to wrap/unwrap/transfers and can accumulate balances on frozen accounts, undermining the intent that operations are blocked for frozen addresses.

— RECOMMENDATION

This is likely acceptable given the trusted nature of these addresses and the claimYield function. However be aware of this circumvention of the freeze list.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

ACKNOWLEDGED

Multiple Initialize Functions Risk

CATEGORY	Warning
LOCATION	USD8.sol
REVIEW	Main Review

— DESCRIPTION

In the base MYieldToOne contract there is an initialize function which has different parameters and therefore a different function signature than the initialize function declared in the USD8 contract.

As a result, there are multiple initialize functions with the initializer modifier available to be called on the USD8 contract. This poses a risk if the wrong initialize function is called, incorrectly initializing the proxy and disabling all other initializers.

— RECOMMENDATION

When deploying the USD8 contract ensure that the initialize function that is implemented on the USD8 contract is the one being invoked based upon the parameters being provided.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-08**

RESOLVED

Outdated MUSD References

CATEGORY	Documentation
LOCATION	USD8.sol
REVIEW	Main Review

— DESCRIPTION

Throughout the USD8 contract in the documentation for the initialize, _beforeWrap, _beforeUnwrap functions mUSD is referenced instead of USD8.

— RECOMMENDATION

Consider updating the comments to refer to USD8 instead of mUSD.

— RESOLUTION

Status: Resolved.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.